Lecture (12)

Binary Search

Linear Search

i = 0	1	2	3	4	5
3	7	0	-2	5	9

```
for (0 → n)
{
    if (arr[i] == Key) {
        return i;
    }
}
```

Key = 2 → absent / Not found

n – size

worst case → n comparisons

⇒ O(n) (Time Comp.)

Binary Search

↳ only for monotonic functions i.e. elements either in dec. or inc. order

Ex:

0	1	2	3	4
3	5	9	13	27

(a) (b)

Key = 13

① mid element

② compare mid element with Key

9 == 13 ✗

③ Now 9 ≠ 13 so which part to go on (a) or (b)

As 13 > 9 so we search in part (b) because values are in increasing order

$\left(\text{mid} = \frac{\text{start} + \text{end}}{2}\right)$

3	4
13	27

mid

$\text{mid} = \frac{3+4}{2} = \frac{7}{2} = 3$

13 == 13 ✓

so present at i = 3

Ex:

Key = 27

0	1	2	3	4	5
3	7	11	13	19	27

(a) mid (b)

$\text{mid} = \frac{0+5}{2} = \frac{5}{2} = 2$

11 == 27 ✗

As 27 > 11 so part (b)

3	4	5
13	19	27

mid

$\text{mid} = \frac{3+5}{2} = \frac{8}{2} = 4$

19 == 27 ✗

So, again right side

5
27

$\text{mid} = \frac{5+5}{2} = 5$

27 == 27 ✓

So, ans = 5

Ex:-

0	1	2	3	4
4	8	16	22	34

(a) mid (b)

$\rightarrow$ mid $= \frac{0+4}{2} = 2$

Key = 4

16 == 4 ✗

As 4 < 16 so part (a)

0	1
4	8

mid

$\rightarrow$ mid $= \frac{0+1}{2} = \frac{1}{2} = 0$

4 == 4 ✓

So ans = 0

Ex:-

0	1	2	3	4	5
5	11	13	17	19	27

mid

$\rightarrow$ mid $= \frac{0+5}{2} = \frac{5}{2} = 2$

key = 25

13 == 25 ✗

25 > 13 so right part

3	4	5
17	19	27

mid

$\rightarrow$ mid $= \frac{3+5}{2} = \frac{8}{2} = 4$

19 == 25 ✗

25 > 19 so right part

5
27

mid

$\rightarrow$ mid $= \frac{5+5}{2} = \frac{10}{2} = 5$

27 == 25 ✗

So, no path left

So, element not found

Note:- We have to keep in mind that in binary search the array should be in increasing order.

Implementation of Binary search

```
# include <iostream>
using namespace std;
int binarySearch (int arr[], int size, int key) {
    int start=0;
    int end = size-1;
    int mid = (start + end) / 2
    while (start <= end) {
        if (arr[mid] == key) {
            return mid;
        }
        else if (key > arr[mid]) {
            start = mid+1;
        }
        else {
            end= mid -1;
        }
        mid = (start + end) / 2;
    }
    return -1;
}
int main () {
    int even[6] = {2, 4, 6, 8, 12, 18};
    int odd[5] = {3, 6, 15, 18, 20};
    int evenIndex = binarySearch (even, 6, 8)
    cout<< "Index of 8 is " << evenIndex << endl;
    int oddIndex = binarySearch (odd, 5, 15)
    cout<< "Index of 15 is " << oddIndex << endl;
    return 0;
}
```

In this error can be there if (start + end) exceeds INT_MAX so this formula can be written as

$$\boxed{mid = start + (end - start)/2}$$

Linear Search vs Binary Search

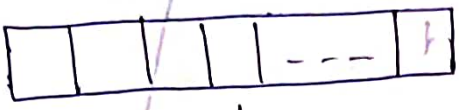
if 1000 values array

In Linear search
↳ 1000 comparisons
(Worst case)
$\Rightarrow O(n)$

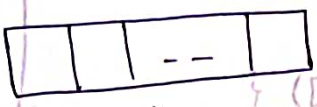
In Binary Search
(Worst case)
$\Rightarrow O(\log n)$

$1000 \rightarrow 500 \rightarrow 250 \rightarrow 125 \rightarrow 62 \rightarrow 31 \rightarrow 15 \rightarrow 7 \rightarrow 3 \rightarrow 1$

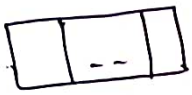
↳ only 10 comparisons



N size $\Rightarrow \frac{N}{2^0}$



$\frac{N}{2}$ size $\Rightarrow \frac{N}{2^1}$



$\frac{N}{4}$ size $\Rightarrow \frac{N}{2^2}$

$\frac{N}{2^K}$ size $\left\{ \frac{N}{2^K} = 1 \right\}$

$\Rightarrow N = 2^K$

$K = \log N$ (comparisons)

So, $O(\log N)$
(Time complexity)

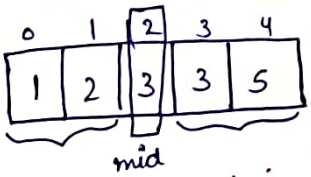
Lecture (13)

Qu- Find first and last position of an element in sorted array

So for sorted array the optimal approach will be Binary Search

So. this problem can be broken into two steps

First occurence or leftmost	last occurence or Rightmost



Key = 3

mid = $\frac{0+4}{2} = 2$

$3 == 3 \rightarrow$ ans == 2 (store)
then go in left part
(end = mid − 1)

```
if (arr[mid] > Key)
    end = mid - 1
if (arr[mid] < Key)
    Start = mid + 1
```

Code:-

```
# include < iostream>
using namespace std;
~~int binary~~
int firstOcc (int arr[], int n, int Key) {
    int s=0, e=n-1;
    int mid = s + (e-s)/2;
    int ans = -1;
    while (s<=e) {
        ~~int~~ if ( arr[mid] == Key) {
            ans= mid
            e= mid-1;
        }
        else if (Key > arr[mid]) {
            s= mid+1;
        }
        else {
            e= mid-1;
        }
        mid = s + (e-s)/2;
    }
    return ans;
}
```

Now for last occurence all the code will be similar to first occurence except in the if (arr[mid] == Key) {

```
            ans= mid;
            s= mid+1;
        }
```

For creating pair :-

```
pair <int, int> p;
p.first = 5
p.second = 7
```

{5, 7}

Qu- I/P ⇒

1	2	3	3	3	3	5

, Find total no of occurence of 3, sorted Array

Ans. first occ. → 2

last occ. → 5

Total no of occ. = (last index − first index) + 1

= 5 − 2 + 1

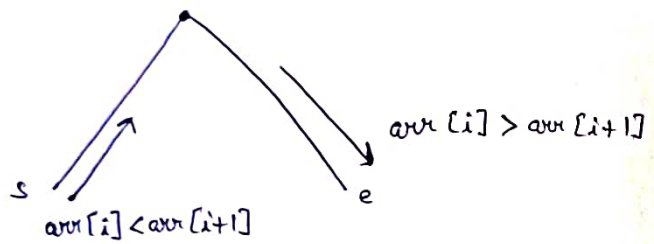
= 4

Qu- Peak index in a Mountain array

Mountain array :- ① arr.length >= 3

② arr[0] < arr[1] < arr[2] <-----<arr[i] > arr[i+1] ---> arr[length

arr[i-1] < arr[i] > arr[i+1]



0	1	2	3
3	4	5	1

mid

mid = $\frac{0+3}{2} = \frac{3}{2} = 1$

```
if (arr[mid] < arr[mid+1]) {
    s = mid+1;
}
else {
    e = mid;
}
```

Why not e = mid-1;

↳ Because if mid was peak element then end will become less than mid

So, e = mid covers both two conditions :- ① arr[mid] > arr[mid+1]

② when mid is at peak

Dry Run:-

0	1	2	3
3	4	5	1

mid

$mid = \frac{0+3}{2} = \frac{3}{2} = 1$

$4 < 5$ i.e. $arr[mid] < arr[mid+1]$

So, $s = mid + 1$

$s = 2$

2	3
5	1

mid

$mid = \frac{2+3}{2} = \frac{5}{2} = 2$

$s < 1$ ✗

so, $e = mid \Rightarrow e = 2$

2
5

mid

$mid = \frac{2+2}{2} = 2$

Now there will be one problem that s will again be compared with it's next element that we had already done earlier

So, for avoiding this in loop we take condn as while $(s < e)$ not $s <= e$ & the answer will be returned as s.

H.W.

Qu- Find pivot in an Array

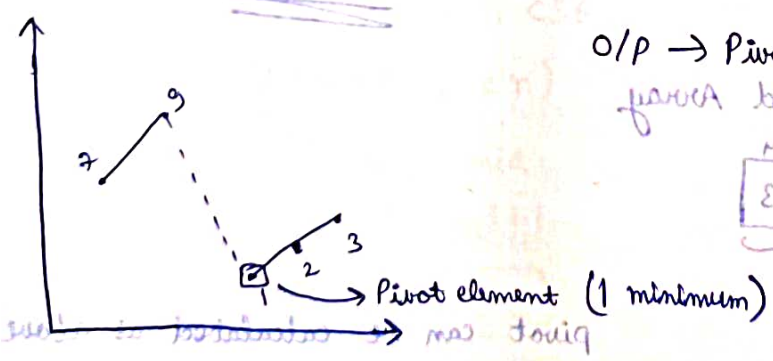
Lecture (14)

Qu- Find pivot in an array. i/p → arr[] ⇒ {1, 2, 3, 7, 9} (sorted)

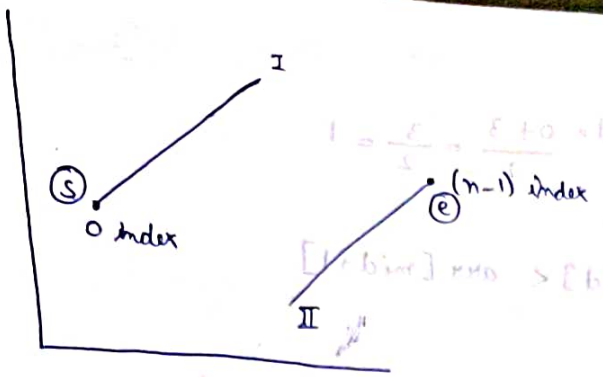
↓

{7, 9, 1, 2, 3} (rotated)

O/P → Pivot?



7	9	1	2	3



arr [mid] >= arr [0] → I st line

↳ s = mid + 1

else → II nd line

↳ e = mid

Dry Run

0	1	2	3	4
3	8	10	17	1

mid

→ mid = $\frac{0+4}{2}$ = 2

arr [mid] >=arr [0] → I st line (lie)

↳ s = mid + 1

3	4
17	1

→ mid = $\frac{3+4}{2}$ = 3

arr [mid] > = arr [0] → I st line

↳ s = mid+1

4
1

mid

arr [mid] > = arr [0] ✗

↳ e = mid

e = 4 & s also = 4

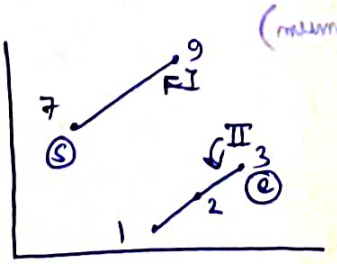
↳ Loop se bahar while (s < e)

return s ⇒ 4

Qu- Search in a rotated and Sorted Array

0	1	2	3	4
7	9	1	2	3

Key = 2



pivot can be calculated as above

E	S	I	e	F

if arr[pivot] ≤ Target ≤ arr[n-1] ↷ B.S. on II

else ↷ B.S. on I

Qu- Square root using Binary search, i/p → 25 , o/p → $\sqrt{25} = 5$

i/p → 27 , o/p → $\sqrt{27} = 5.\ldots = 5$

(only integer part)

Ans. i/p → 27

o/p → [0 ——— 27] ↷ 0, 1, 2, 3, ---, 25, 26, 27

Search Space

i/p → 92

o/p → [0 ——— 92]

n = 36

[0 ——— 18 ——— 36] ↷ $mid = \frac{36}{2} = 18$

mid

18 X 18 = 324 so right part will be of no use

i.e. if (mid * mid > n)

↳ e = mid-1

Cond^n ⇒ while (s<=e)

[0 — 8 — 17]

mid

if (mid * mid) not in int range

Take long long int

8 X 8 = 64 > 36

e = mid-1

[0 — 3 — 7]

mid

3 X 3 = 9 < 36

if (mid * mid < n)

↳ ans = mid

s = mid+1

[4 — 5 — 7]

mid

5 X 5 = 25 < 36

ans = 5

s = mid + 1 = 6

[6 | 7]

mid

6 X 6 = 36 = 36

if (mid * mid == n)

return mid

Qu- Now also finding float part of square root

$\sqrt{37} \rightarrow 6.12$

↑ Ve to nikal jayega

$6 \xrightarrow{+} 0.1 \Rightarrow 6.1 \Rightarrow (6.1)^2 < 37$ ↷ ans save

$6.2 \Rightarrow (6.2)^2 < 37$ ↷ ans save

$6.3 \Rightarrow (6.3)^2 < 37$ (False)

Now, $6.2 \xrightarrow{+} 0.01 \Rightarrow (6.21)^2 < 37$ ↷ save ans

$(6.22)^2 < 37$ ↷ save ans

$(6.23)^2 < 37$ (False)

Now, $6.22 \xrightarrow{+} 0.001 \rightarrow$ Process

<u>Code for this :-</u>

```
double morePrecision (int n, int precision, int temp Sol) {
    double ans = temp Sol
    double factor = 1;

    for (int i=0; i<precision; i++) {
        factor = factor/10;
        for (double j=ans; j*j < n; j=j+factor) {
            ans = j;
        }
    }
    return ans;
}
```

<u>Lecture (15)</u>

Qu- Book Allocation Problem,

Books arr [10 | 20 | 30 | 40]

① Each student gets atleast one book

② Each book should be allocated to a student

③ Book allocation should be in a contiguous manner

arr[i] = number of pages

Total students = m

You have to allocate the book to m students such that the maximum number of pages assigned to a student is minimum.

Sol^n

0	1	2	3
10	20	30	40

$n = 4$ (Books)

$m = 2$ (Students)

Case ① ⇒ $\underbrace{10}_{10} \mid \underbrace{20 \quad 30 \quad 40}_{90}$ ⇒ max. = 90 pages

Case ② ⇒ $\underbrace{10 \quad 20}_{30} \mid \underbrace{30 \quad 40}_{70}$ ⇒ max. = 70 pages

Case ③ ⇒ $\underbrace{10 \quad 20 \quad 30}_{60} \mid \underbrace{40}_{40}$ ⇒ max. = 60 pages

min. ⇒ 60 pages

Answer

10	20	30	40

$m = 2$

min. → 0

max. → 100

ans → search space

0 —— $\boxed{50}$ —— 100

(s) mid (e)

mid = 50

is Possible Solution (mid) → $m = 2$

10 20 | 30 | 40

I → $\boxed{10 + 20} + 30$ ✗

II → $\boxed{30} + 40$ ✗

III → $\boxed{40}$

Three students but $m = 2$

so 50 is not a possible sol^n

To 2 students 50 bhi nhi bana paa rahe hai to matlab 50 ke left me To nhi jaayenge because 2 students 50 hi nahi bana paa rahe To usse choti values bhi nhi bana paayenge

so, $s = mid + 1$ (when no sol^n exist)

Now, 51 —— $\boxed{75}$ —— 100

mid

10 20 30 40

I → $\boxed{10 + 20 + 30} + 40$ ✗

II → 40

Possible sol^n

Now, agar 75 possible sol^n hai to uske aage vaale to honge hi

So, $e = mid - 1$ (when sol^n exists)

& ans ko store bhi Kara lena

51 — $\boxed{62}$ — 74 $\curvearrowright$ $mid = \frac{51+74}{2} = \frac{125}{2} = 62$

mid

10 20 30 | 40

I $\Rightarrow$ $\boxed{10+20+30}$ + 40 α

II $\Rightarrow$ $\boxed{40}$

Possible sol^n

So, $e = mid - 1$

51 — $\boxed{56}$ — 61 $\curvearrowright$ $mid = \frac{51+61}{2} = 56$

mid

10 20 | 30 | 40

I $\rightarrow$ $\boxed{10+20}$ + 30 α

II $\rightarrow$ $\boxed{30}$ + 40 α

III $\rightarrow$ $\boxed{40}$

No sol^n

As $m = 2$ only

So, $s = mid + 1$

57 — $\boxed{59}$ — 61 $\curvearrowright$ $mid = \frac{57+61}{2} = 59$

mid

10 20 | 30 | 40

I $\rightarrow$ $\boxed{10+20}$ + 30 α

II $\rightarrow$ $\boxed{30}$ + 40 α

III $\rightarrow$ $\boxed{40}$

No sol^n

As $m = 2$ only

So, $s = mid + 1$

$\boxed{60}$ $\boxed{61}$

mid

10 20 30 | 40

I → $\boxed{10 + 20 + 30}$ + 40 } Possible sol^n

II → $\boxed{40}$

So, e = mid − 1 But cond^n is s <= e so loop se bahar

so, ans = 60

Code:-

```
int allocateBooks (vector<int> arr, int n, int m) {

    int s = 0;
    int sum = 0;
    for (int i = 0; i< n; i++) {
        sum += arr[i];
    }

    int e = sum;
    int ans = -1;
    int mid = s + (e-s)/2;

    while (s <= e) {
        if (isPossible (arr, n, m, mid)) {
            ans = mid;
            e = mid - 1;
        }
        else {
            s = mid + 1;
        }

        mid = s + (e-s)/2;
    }

    return ans;
}
```

Now, Function for possible:-

```
bool isPossible (vector<int> arr, int n, int m, int mid) {
    int StudentCount = 1;
    int pageSum = 0;
    for (int i=0; i<n; i++) {
        if (pageSum + arr[i] <= mid)
        {
            pageSum += arr[i];
        }
        else {
            StudentCount ++;
            if (StudentCount > m || arr[i] > mid) {
                return false;
            }
            ~~pageSum = 0~~;
            pageSum = arr[i];
        }
    }
    return true;
}
```

Qu- Painter's Partition Problem

arr → { - - - - - - }N K painters

1 unit Board → 1 unit time

Painter → contiguous section

} Boards ka partition karna hai paint karne ka such that time required should be minimum

Sol^n

5	5	5	5

K = 2

0 —— $\boxed{10}$ —— 20
mid

5 5 | 5 5

I → $\boxed{5+5}$ + 5 ✗

II → $\boxed{5+5}$

} Possible sol^n exists
ans = 10
e = mid − 1

0 — $\boxed{4}$ — 9

mid

S S S S

I → S α

Not possible

S = mid + 1

S — $\boxed{7}$ — 9

mid

S S S S

I → $\boxed{S}$ + S α

II → $\boxed{S}$ + S α

III →

No sol^n

S = mid + 1

$\boxed{8}$ $\boxed{9}$

mid

S S S S

I → $\boxed{S}$ + S α

II → $\boxed{S}$ + S α

III →

no sol^n

S = mid + 1

$\boxed{9}$

mid

S S S S

I → $\boxed{S}$ + S α

II → $\boxed{S}$ + S α

III →

no sol^n

S = mid + 1

As S should be < = e

so, out of loop

So, answer should be that which was stored previously i.e. 10

↳ This question is same as previous question

Qu- Aggressive Cows

arr ⇒ { - - - - - - }N

arr[i] = position of stall

K → number of cows

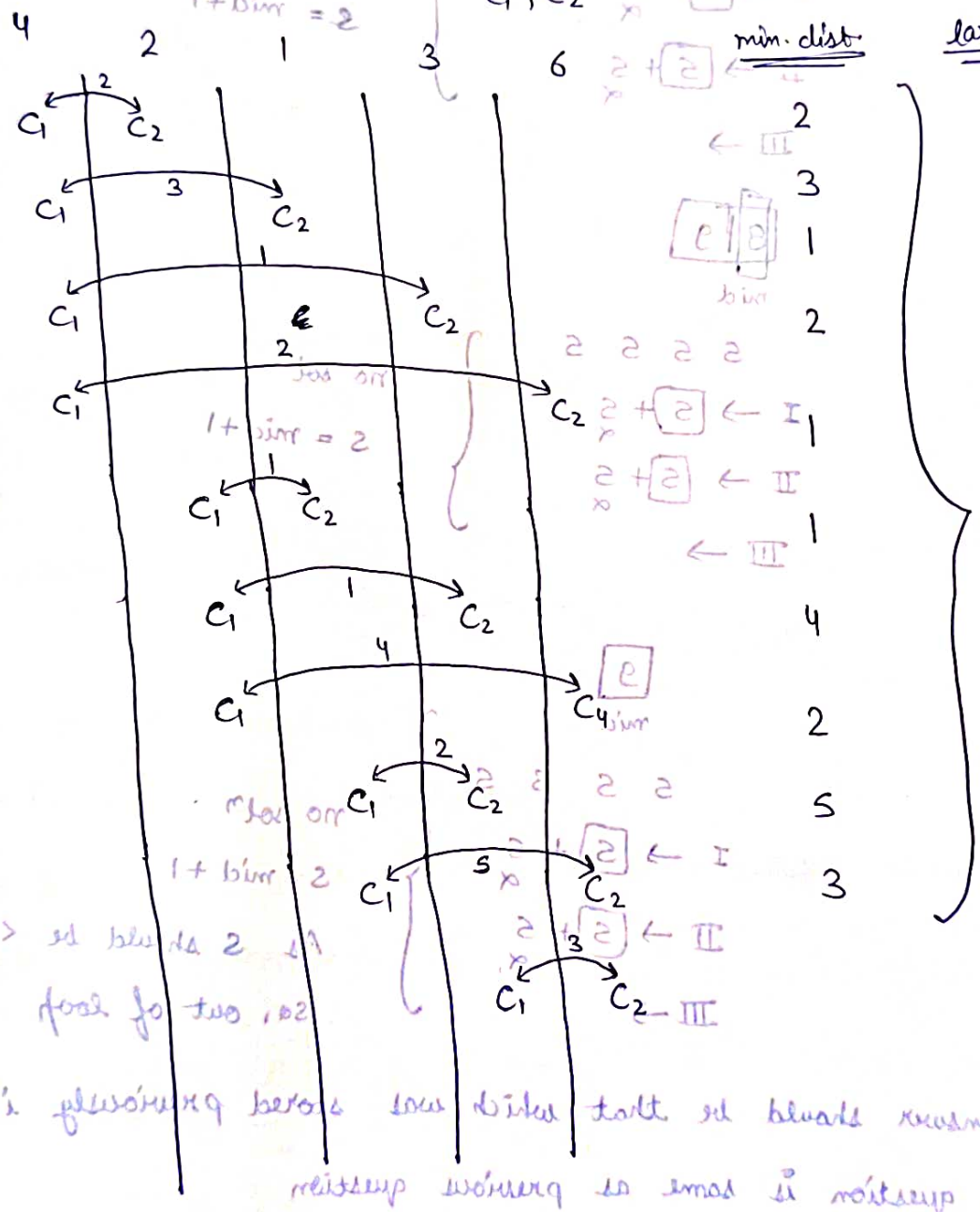
To prevent the cows from hurting each other, you need to assign the cows to the stalls such that the minimum distance between any two of them is as large as possible. Return the largest minimum distance.

Solⁿ

arr ⇒ {4, 2, 1, 3, 6}N ← stall positions

K = 2

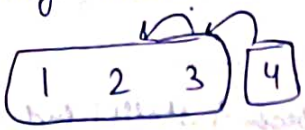
C1, C2



4 2 1 3 6

Search Space

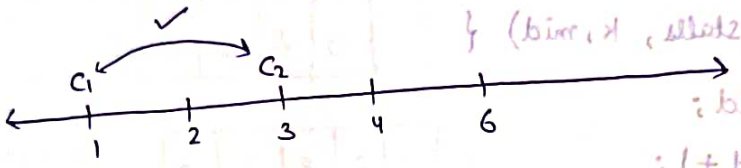
Why Binary Search?



Agar ek mid ke neglect kone se poora ek part neglect ho jaaye To un questions me Binary Search use Karte Hai

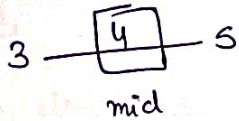
4	2	1	3	6

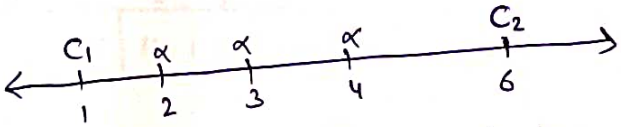
Search space
- min → 0
- max. → maxi − mini
  6 − 1 = 5



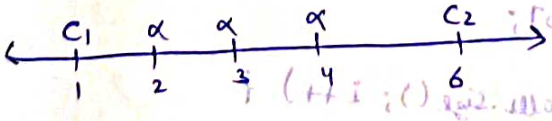
$C_1 \longleftrightarrow C_2$ ✗

2 >= 2 ⇒ True } Possible ans = 2
s = mid + 1 (Because right part me jaayenge max. ke liye)





s >= 4 → Possible
ans = 4
s = mid + 1



s >= 5 → Possible
s = mid + 1 (But in this case s>e)
So out of loop

Code:-

```
int aggressiveCows (vector<int> & stalls, int k) {
    Sort (v.begin(), v.end());
    int s = 0;
    int maxi = -1;
    for (int i=0; i< stalls.size(); i++) {
        maxi = max (maxi, stalls[i]);
    }
    int e = maxi;
    int ans = -1;
    int mid = s+ (e-s)/2;
    while (s<=e) {
        if (isPossible (stalls, k, mid) {
            ans = mid;
            s = mid+1;
        }
        else {
            e= mid-1;
        }
        mid = s + (e-s)/2;
    }
    return ans;
}
```

→ v = stalls
so write stalls.begin & stalls.end

Function for possible:-

```
bool isPossible (stalls, k, mid) {
    int cowCount = 1;
    int lastPos = stalls[0];
    for (int i=0; i < stalls.size(); i++) {
        if (arr[i] - lastPos >= mid) {
            cowCount++;
```

```
        if (cowCount == K) {
            return true;
        }
        lastPos = stalls[i];
    }
}
return false;
}
```

H.W.

↳ In description of lecture 15

Lecture 16

Selection Sort

arr [] = | 1 | 7 | 9 | 2 | 3 | 0 |

sorted array ⇒ | 0 | 1 | 2 | 3 | 7 | 9 |

→ inc. order

What?

↳ different Rounds/passes

↳ Smallest element lake usko right jagah par place kar dete hai

Ex:- arr [] ⇒

0	1	2	3	4
64	25	12	22	11

Round 1:- $\overset{i=0}{64}$ 25 12 22 (11) ⇒ 11 25 12 22 64

swap

Round 2:- 11 $\overset{i=1}{25}$ (12) 22 64 ⇒ 11 12 25 22 64

swap

Round 3:- 11 12 $\overset{i=2}{25}$ (22) 64 ⇒ 11 12 22 25 64

swap

Round 4:- 11 12 22 $\overset{i=3}{(25)}$ 64 ⇒ 11 12 22 25 64

Total Round ⇒ $n - 1$

Code:-

```
for (int i=0; i< n-1; i++) {
    int minIndex = i;
    for (int j= i+1; j<n; j++) {
        if (arr[j] < arr[minIndex]) {
            minIndex = j;
        }
    }
    swap (arr[minIndex], arr[i]);
}
```

```
for (int i=0; i<n-1; i++) {
    for (int j=i+1; j<n; j++) {
        if (arr[j] < arr[i]) {
            swap (arr[j], arr[i]);
        }
    }
}
```

Space Complexity ⇒ O(1)

Time Complexity

$1 + 2 + 3 + \dots + (n-2) + (n-1)$

$= \frac{n(n-1)}{2} \Rightarrow \frac{n^2 - n}{2}$

$\Rightarrow O(n^2)$

Time Complexity

Best Case ⇒ Array already Sorted

Sort Ho ya no ~~ka~~ comparison to tab hi honge or utne hi honge

So, Best case = Worst case = $O(n^2)$

When to use Selection sort

↳ When array / Vector / list size is small

H.W.

① Draw flowchart of selection sort

② Is selection sort a stable or unstable algorithm

Lecture - 17

Bubble Sort

What ?

Ex:-

10	1	7	6	14	9

Round 1 :- 10 1 7 6 14 9

swap

~~Round 2~~ 1 10 7 6 14 9

swap

~~Round 3~~ 1 7 10 6 14 9

swap

~~Round 4~~ 1 7 6 10 14 9

Ignore

~~Round~~ 1 7 6 10 14 9

swap

1 7 6 10 9 14

Sorted

So, in Round ① first largest element comes in right place

& Total rounds = $n-1$

Round 2 :- 1 7 6 10 9 | 14

ignore

1 7 6 10 9 | 14

swap

1 6 7 10 9 14

Ignore

1 6 7 10 9 14

swap

1 6 7 9 10 14

In Round ②, second largest element sorted

Round 3 :- 1 6 7 9 | 10 14

Ignore

1 6 7 9 10 14 — 3rd largest sorted

Ignore

1 6 7 9 10 14

Ignore

1 6 7 9 10 14

Round 4 :- 1 6 7 | 9 10 14

Ignore

4th largest sorted

1 6 7 9 10 14

Ignore

1 6 7 9 10 14

Round 5 :- 1 6 | 7 9 10 14

Ignore

All elements sorted

1 6 7 9 10 14

In Bubble sort har i^{th} Round me aap i^{th} largest element ko uske right place par pahucha dete Ho

Code:-

```
for (int i=1; i<n; i++) {
    for (int j=0; j<n-i; j++) {
        if (arr[j] > arr[j+1]) {
            swap (arr[j], arr[j+1]);
        }
    }
}
```

Time Complexity

Round 1 → (n-1) comparisons
2 → (n-2)
3 → (n-3)
⋮
(n-1) → 1

$$1 + 2 + 3 + \cdots + (n-2) + (n-1)$$
$$= \frac{n(n-1)}{2}$$
$$\Rightarrow T.C. = O(n^2)$$

Space Complexity ⇒ O(1)

Best case ⇒ Already sorted array
↳ if no swaps are there in first round
So, complexity = O(n)

H.O.
① Bubble sort is stable or unstable algorithm
② In-place sort ?
③ GFG quiz in description

Lecture-18

Insertion Sort

Ex:- arr[] ⇒

4	12	11	20

① → 4
② → 4 12
③ → 4 11 12
④ → 4 11 12 20

Compare Karate jaana Hai Har next element ko or usi according place Karna Hai

Ex-

0	1	2	3	4	5	6
10	1	7	4	8	2	11

Round 1:- i=0 pass me rakh liya or compare Karna start Kiya i=1 se
1 < 10 → So left
But left me to koi jaga hi nhi hai to Hame right shift Karna padega 1 place

1 10 | 7 4 8 2 11

Round 2:- $(i=2)$

$7 < 10 \rightarrow$ left

$7 > 1 \rightarrow$ right

1 7 10 | 4 8 2 11

Round 3:- $(i=3)$

$4 < 10 \rightarrow$ left

$4 < 7 \rightarrow$ left

$4 > 1 \rightarrow$ right

1 4 7 10 | 8 2 11

Round 4:- $(i=4)$

$8 < 10 \rightarrow$ left

$8 > 7 \rightarrow$ right

1 4 7 8 10 | 2 11

Round 5:- $(i=5)$

$2 < 10 \rightarrow$ left

$2 < 8 \rightarrow$ left

$2 < 7 \rightarrow$ left

$2 < 4 \rightarrow$ left

$2 > 1 \rightarrow$ right

1 2 4 7 8 10 | 11

Round 6:- $(i=6)$

$11 > 10 \rightarrow$ right

1 2 4 7 8 10 11 **Sorted**

Total Rounds = $n-1$

Code:-

```
for (int i=1; i<n; i++) {
    int temp = arr[i];

    int j = i-1;
    for (; j>=0; j--) {
        if (arr[j] > temp) {
            arr[j+1] = arr[j];
        }
        else {
            break;
        }
    }
    arr[j+1] = temp;
}
```

Why Insertion Sort?

- Adaptable & Stable Algorithm
- Good for small n

Space Complexity ⇒ O(1)

Time Complexity

$1 + 2 + \_ \_ \_ + (n-1) \Rightarrow O(n^2)$

Best case → array already sorted

↳ Total (n-1) comparisons

T.C. best case ⇒ O(n)

Lecture (19)

C++ STL

- Algorithms
  - Binary search
  - Lower/Upper bound
  - min/max
  - reverse/rotate
  - sort/swap etc.
- Containers (Pre built Data Structures)
  - Sequence Containers
    - Array
    - Vector
    - Deque
    - List
    - Forward list
  - Container Adaptors
    - Stack
    - Queue
    - Priority Queue
  - Associative Containers
    - Set
    - Map
    - Multiset
    - Multimap
  - Unordered Associative
    - Unordered set
    - Unordered map
    - Unordered multiset
    - Unordered multimap

Array

```
# include < iostream >
# include < array >

using namespace std;

int main () {

    int basic [3] = {1, 2, 3};

    array < int, 4 > a = {1,2,3,4};

    int size = a.size();

    for (int i=0; i< size; i++) {

        cout << a [i] << endl;
    }

    cout << " Element at 2nd Index -> " << a.at (2) << endl;
    cout << " Empty or not -> " << a. empty () << endl;
    cout << " First element -> " << a. front () << endl;
    cout << " Last element -> " << a. back () << endl;
```

Output:-

```
1
2
3
4
Element at 2nd index -> 3
Empty or not -> 0
First element -> 1
Last element -> 4
```

Vector (Dynamic Array)

Capacity ⇒ memory allocated to array
Size ⇒ size of array

```
# include <iostream>
# include <vector>
using namespace std;

int main() {

    vector<int> V;
    cout << " Capacity -> " << V. capacity () << endl;

    V. push_back (1);
    cout << " Capacity -> " V. capacity () << endl;

    V. push_back (2);
    cout << " Capacity -> " V. capacity () << endl;

    V. push_back (3);
    cout << " Capacity -> " V. capacity () << endl;
    cout << " Size -> " << V. size () << endl;

    cout << " Element at 2nd Index " << V. at(2) << endl;

    cout << " front " << V. front() << endl;
    cout << " back " << V. back () << endl;

    cout << " before pop " << endl;
    for (int i: V) {
        cout << i << " ";
    }
    cout << endl;

    V. pop_back();

    cout << " after pop " << endl;
    for (int i: V) {
        cout << i << " ";
    }
```

```
cout << endl;
cout << " before clear size " << v.size() << endl;
v.clear();
cout << " after clear size " << v.size() << endl;
vector <int> a(5,1);   → all 5 elements initialized by 1
cout << " print a " << endl;
for (int i:a) {
    cout << i << " ";
}
cout << endl;
vector<int> p(a);   → For copying
cout << " print p " << endl;
for (int i:p) {
    cout << i << " ";
}
}
```

Output:-

```
Capacity -> 0
Capacity -> 1
Capacity -> 2
Capacity -> 4
size -> 3
Element at 2nd Index 3
front 1
back 3
before pop
1 2 3
after pop
1 2
before clear size 2
after clear size 0
```

print a

1 1 1 1 1

print p

1 1 1 1 1

Deque (Doubly ended queue)

- Dono end par deletion or insertion kar sakte hai
- Dynamic
- Random access possible

```
# include <iostream>
# include <deque>
using namespace std;

int main() {

    deque <int> d;

    d.push_back (1);
    d.push_front (2);
    for (int i : d) {
        cout << i << "  ";
    }
    cout << endl;
    d.pop_front();
    for (int i : d) {
        cout << i << "  ";
    }
    cout << endl;
    d.push_front (3);
    cout << " print first index element -> " << d.at (1) << endl;
    cout << " front " << d.front () << endl;
    cout << " back " << d.back () << endl;
    cout << " before erase " << d.size () << endl;
    d.erase ( d.begin (), d.begin() + 1);
```

```
    cout << "after erase" << d.size() << endl;
    for (int i: d) {
        cout << i << endl;
    }
}
```

Output

```
2  1
1
Print first index element -> 1
front 3
back 1
before erase 2
after erase 1
1
```

List

```
# include <iostream>
# include <list>
using namespace std;
int main () {
    list<int> l;
    l.push_back(1);
    l.push_front(2);
    for (int i: l) {
        cout << i << " ";
    }
    cout << endl;
    l.erase(l.begin());
    cout << "after erase" << endl;
    for (int i: l) {
        cout << i << " ";
    }
}
```

Output:-

```
2  1
after erase
1
```

Stack (Pile of plates se assume Karo)

↳ last in first out concept

```
# include < iostream >
# include < stack >
using namespace std;
int main () {
    Stack<string> s;
    s. push ("love");
    s. push ("babbar");
    s. push ("Kumar");
    cout << "Top element ->" << s. top () << endl;
    s. pop ();
    cout << "Top element ->" << s. top () << endl;
    cout << " size of stack " << s. size() << endl;
    cout << "Empty or not " << s. empty () << endl;
}
```

Output:-

Top element -> Kumar
Top element -> babbar
size of stack 2
Empty or not 0

Queue

↳ First in first out

```
# include < iostream >
# include < queue >
using namespace std;
int main () {
    queue < string > q;
    q. push ("love");
    q. push ("Babbar");
    q. push ("Kumar");
    cout << " First element " << q. front() << endl;
    q. pop ();
    cout << " First element " << q. front () << endl;
}
```

Output:-

First element love
First element Babbar

Priority Queue

- First element always greatest
- Default Priority queue ⇒ max. heap

```
# include <iostream>
# include <queue>

using namespace std;

int main () {

    // max heap

    priority_queue <int> maxi;

    // min heap

    priority_queue <int , vector<int>, greater<int>> mini;

    maxi.push (1);
    maxi.push (3);
    maxi.push (2);
    maxi.push (0);
    int n = maxi.size()
    for ( int i=0; i< ~~maxi.size()~~ n; i++) {

        cout << maxi.top () << " ";

        maxi.pop ();

    }
    cout << endl;

    mini.push (5);
    mini.push (1);
    mini.push (0);
    mini.push (4);
    mini.push (3);

    int m = mini.size ();
```

Output:-

3 2 1 0

0 1 3 4 5

Empty or not ? → 1

```
for (int i=0; i<m; i++) {
    cout << mini.top() << " ";
    mini.pop();
} cout << endl;

cout << " Empty or not ? ->" << mini.empty() << endl;
}
```

Set

- Unique element si store honge
- Elements return in sorted order

```
# include <iostream>
# include <set>

using namespace std;

int main() {
    set<int> s;

    s.insert(5);
    s.insert(5);
    s.insert(5);
    s.insert(1);
    s.insert(6);
    s.insert(6);
    s.insert(0);
    s.insert(0);
    s.insert(0);

    for (auto i : s) {
        cout << i << endl;
    } cout << endl;

    s.erase(s.begin());

    for (auto i : s) {
        cout << i << endl;
    }
```

Output:-

```
0
1
5
6

1
5
6

5 is present or not -> 1

5 6
```

```
cout << endl;
cout << " s is present or not -> " << s.count(s) << endl;

Set<int> :: iterator itr = s.find(s);

~~cout << " Value~~

for (auto it = itr; it != s.end(); it++) {

        cout << *it << " ";

    } cout << endl;

}
```

Insert, find, erase, count ⇒ T.C. ⇒ O(log n)

Map

→ 1 Key 1 hi value ko point karegi
→ sorted hota hai (But in case of unsorted map ye sorted nahi hota)

```
# include <iostream>

# include <map>

using namespace std;

int main () {

    map<int, string> m;

    m[1] = " vaibhav ";

    m[2] = " chitransh ";

    m[13] = " rishi ";
    m.insert ({5, " bheem "});
    for (auto i:m) {
        cout << i.first << endl;
    }

    for (auto i:m) {
        cout << i.first << " " << i.second << endl;
    }

}
```

Output:-

```
1
2
5
13
1 vaibhav
2 chitransh
5 bheem
13 rishi
```

STL Algorithms

```
#include<iostream>
#include<algorithm>
#include<vector>
using namespace std;

int main() {
    vector<int> v;
    v.push_back(1);
    v.push_back(3);
    v.push_back(6);
    v.push_back(7);

    cout<<"Finding 5 ->"<<binary_search(v.begin(), v.end(), 5)<<endl;
    cout<<"Finding 6 ->"<<binary_search(v.begin(), v.end(), 6)<<endl;
    cout<<"lower bound ->"<<lower_bound(v.begin(), v.end(), 6)-v.begin()<<endl;
    cout<<"Upper bound ->"<<upper_bound(v.begin(), v.end(), 4)-v.begin()<<endl;

    int a=3;
    int b=5;
    cout<<"max ->"<<max(a, b)<<endl;
    cout<<"min ->"<<min(a, b)<<endl;
    swap(a, b);
    cout<<"a ->"<<a<<endl;

    string abcd="abcd";
    reverse(abcd.begin(), abcd.end());
    cout<<"string ->"<<abcd<<endl;
    rotate(v.begin(), v.begin()+1, v.end());
    cout<<"after rotate"<<endl;
    for (int i:v) {
        cout<<i<<" ";
    } cout<<endl;
}
```

Output:-

```
Finding 5 -> 0
Finding 6 -> 1
lower bound -> 2
Upper bound -> 2
max -> 5
min -> 3
a -> 5
string -> dcba
after rotate
3 6 7 1
```

Lecture 20

Qu- Reverse the Array , I/P ⇒ {1, 3, 2, 4} O/P ⇒ {4, 2, 3, 1}

arr[] ⇒

0	1	2	3	4
11	7	3	12	4
s				e

swap

4 7 3 12 11

s e

4 12 3 7 11

s e

⇒ 4 12 3 7 11 (Reversed Array)

```
# include <iostream>
# include <vector>
using namespace std;
vector <int> reverse (vector<int> v) {
        int s=0, e= v.size() -1;
        while (s<=e) {
                swap (v[s], v[e]);
                s++;
                e--;
        }
        return v;
}
void print (vector<int> v) {
        for (int i=0; i<v.size(); i++) {
                cout<<v[i]<<" ";
        }
        cout<<endl;
}
int main () {
        vector<int> v;
```

```
    v.push_back(11);
    v.push_back(7);
    v.push_back(3);
    v.push_back(12);
    v.push_back(4);
    vector<int> ans = reverse(v);
    print(ans);
    return 0;
}
```

Qu- Given an array arr of integers and a position m. You have to reverse the array after that position.

```
void reverseArray(vector<int> &arr, int m) {
    int s = m+1, e = arr.size() -1;
    while (s<=e) {
        swap(arr[s], arr[e]);
        s++;
        e--;
    }
}
```

Leetcode 88

Qu- Merge Sorted Array, arr1[] = {1, 3, 5, 7, 9}, arr2[] = {2, 4, 6}

O/P => {1, 2, 3, 4, 5, 6, 7, 9}

```
# include <iostream>
# include <vector>
using namespace std;

void merge(int arr1[], int n, int arr2[], int m, int arr3[]) {
    int i=0, j=0;
    int k=0;
```

```
while (i<n && j<m) {
    if (arr1[i] < arr2[j]) {
        arr3[k] = arr1[i];
        k++;
        i++;
    }
    else {
        arr3[k] = arr2[j];
        k++
        j++;
    }
}

// Copy first array ke (remaining element ko

while (i<n) {
    arr3[k] = arr1[i];
    k++;
    i++;
}

// Copy second array ke remaining element ko

while (j<m) {
    arr3[k] = arr2[j]
    k++;
    j++;
}
}

void print (int ans[], int n) {
    for (int i=0; i<n; i++) {
        cout << ans[i] << " ";
    }
    cout << endl;
}
```

```
int main() {
    int arr1[5] = {1,3,5,7,9};
    int arr2[3] = {2,4,6};
    int arr3[8] = {0};
    merge (arr1, 5, arr2, 3, arr3);
    print (arr3, 8);
    return 0;
}
```

Leetcode 283

Qu- Move zeroes, I/P ⇒ {0, 1, 0, 3, 12} O/P ⇒ {1, 3, 12, 0, 0}

I/P ⇒ {2, 0, 1, 3, 0, 0, 0} O/P ⇒ {2, 1, 3, 0, 0, 0, 0}

arr[] ⇒

0	1	0	3	12	0	

```
int i=0;    // indicating non-zero value
for (int j=0; j<size; j++) {
    if (arr[j] != 0) {
        swap (arr[j], arr[i]);
        i++;
    }
}
```

My sol^n

```
for (int i=0; i<n; i++) {
    for (int j=0; j<n-i-1; j++) {
        if (arr[j] == 0) {
            swap (arr[j], arr[j+1]);
        }
    }
}
```

Lecture (21)

Leetcode 189

Qu- Rotate Array , I/P ⇒ {1, 7, 9, 11} ⇒ O/P ⇒ {9, 11, 1, 7}

↳ K=2

Thought process

% n ⇒ o/p or ans = [0 to (n-1)]

(n+K) % n = K

arr[(i+K) % n] = arr[i]

↳ cyclic way me K position se shift Kardega

if K=4 ⇒ fully rotate (~~same~~ becomes original)

if K=5 ⇒ (4+1) ⇒ 1 rotation

if K=6 ⇒ (4+2) ⇒ 2 rotation

⇒ so, $\boxed{K = K \% n}$

```
void rotate (vector<int> & nums , int K) {
    vector<int> temp (nums.size());

    for (int i=0; i< nums.size(); i++) {
        temp [ (i+K) % nums.size()] = nums[i];
    }

    nums = temp;
}
```

Leetcode 1752

Qu- Check if array is sorted and rotated.

Case 1 :- {1, 2, 3, 4, 5} Sorted

Case 2 :- { 3, 4, 5, 1, 2} Sorted & Rotated

Case 3 :- { 3, 5, 7, 1, 6} Nothing

Case 4 :- { 1, 1, 1, 1} All equal

True ⇒ 1 pair exist nums[i-1] > nums[i] (in cyclic order)

```
bool check (vector <int> & nums) {
    int count=0;

    for (int i=1; i<n; i++) {
        if (nums [i-1] > nums [i]) {
            count++;
        }
    }
```

```
// Condⁿ for cyclic one
if (nums[nums.size()-1] > nums[0]) {
        count++;
}
return count <= 1;
}
```

Qu- Sum of Two arrays,

arr 1 = | 1 | 2 | 3 | 4 |

arr 2 = | 6 |

O/P ⇒

```
  1 2 3 4
    +   6
  -------
  | 1 | 2 | 4 | 0 |
```

Case I :-

```
  1 2 3 | 4 |
        | 6 |
  -------
  1 2 4 0
```

Case 2 :-

```
  1 2 3 | 6 |
        | 4 |
  -------
```

Case 3 :-

```
  9 9 9
Carry
  9 9 9
  -------
  | | | | |
```

		i
1	1	

		j
2	3	

Sum = 1+3 = 4 , Carry = 0

Carry = Sum / 10

Sum = Sum % 10

arr [K] = Sum

```
int i = n-1;
int j = m-1;
int carry = 0;
while (i>=0 && j>=0) {
    int Val 1 = a[i];
    int Val 2 = b[j];
    int sum = Val 1 + Val 2 + carry;
    carry = sum / 10;
    Sum = Sum % 10;
```

```
    ans.push_back(sum);
    i--;
    j--;
}

// first case
while (i>=0) {
    int sum = a[i] + carry;
    carry = sum/10;
    sum = sum % 10;
    ans.push_back(sum);
    i--;
}

// second case
while (j>=0) {
    int sum = b[j] + carry;
    carry = sum/10;
    sum = sum % 10;
    ans.push_back(sum);
    j--;
}

// third case
while (carry != 0) {
    int sum = carry;
    carry = sum/10;
    sum = sum % 10;
    ans.push_back(sum);
}
// print ans
reverse(ans);
return reverse(ans);
```

Function for reverse:-

```
Vector <int>  reverse (vector<int>&v) {

    int s=0;
    int e = v.size() - 1;

    while (s<e) {

        swap (v[s++], v[e--]);

    }

    return v;

}
```

Lecture (22)

char a = 'z' [z]

Single character a

For multiple character ⇒ char Arrays , strings

strings in C++ ⇒ 1- Dimensional char arrays

strings ⇒

a	b	c	d	e	f

character array

↳ char ch[10];

Taking input & output

char name [20];

i/p ⇒ cin >> name;

B	a	b	b	a	r	\0	---	
0	1	2	3	4	5	6	---	19

'\0' → null character (automatically add ho jaata hai)

↳ use as a terminator

↳ Jisse mujhe pata lage string end kaha ho rahi hai

O/P ⇒ cout << name;

```
# include <iostream>
using namespace std;

int main () {

    cout << " Enter your name" << endl;

    char name[20];
    cin >> name;
    cout << " Your name is " << name;

}
```

Output:-

Enter your name

Vaibhav Chitransh

Your name is Vaibhav

Now, why full name is not showing?

cin → execution stop when space " _ ", tab \t, enter \n

If I/p ⇒ Ba\0bbar

↳ How ⇒ name[2] = '\0';

So, O/P ⇒ Ba

Now, array

B	a	\0	b	a	r	\0	____

garbage values

Qu- Find the length of string

```
int getLength (char name[]) {
        int count = 0;
        for (int i=0; name[i] != '\0'; i++) {
                count ++;
        }
        return count;
}

int main () {
    char name [20];
    cout << " Enter your name" << endl;
    cin >> name;
    cout << " Length : " << getLength (name) << endl;
    return 0;
}
```

Leetcode 344

Qu- Reverse a string

```
void reverse (char name [], int n) {
    int s = 0;
    int e = n-1;
    while (s<=e) {
        swap (name [s++], name [e--]);
    }
}
```

length lena main code mai

↳ Rest code is same as previous ques.

Qu- Check Pallindrome for a string (Consider alphabets & numbers only & ignore symbols and white spaces)

```
bool checkPalindrome (char a[], int n) {
    int s=0;
    int e=n-1;
    while (s<=e) {
        if (a[s] != a[e]) {
            return 0;
        }
        else {
            s++;
            e--;
        }
    }
    return 1;
}
```

Changing Uppercase to lowercase ⇒ temp = ch - 'A' + 'a' ⇒ resultant char

↳ Use this if previous question has one more cond^n which says string is not case-sensitive

H.W.

Leetcode 125